# Boogle Dox — Product Requirements & Product Thinking

**Project:** Ajaia Full Stack Product Engineer take-home **Timebox:** 4–6 hours **Status (2026-11-04):** Core requirements implemented and verified; Collaborative Drafts planned, not built. Every status below is backed by a check recorded in `DEVELOPMENT_LOG.md`.

> This document began as my planning document (archived unchanged in `docs/archive/chatgpt-planning-pack/`). I kept my original reasoning and added what changed once the product was built.

## Why I chose this product direction

I started by asking myself what I actually rely on when I use an online document editor. For me, being able to name a document, find the familiar File, Edit, View, Insert, Format, and Tools menus, format text, undo mistakes, and share a document are all important. I wanted Boogle Dox to feel recognizable right away, without trying to rebuild every Google Docs feature.

Formatting and usability matter to me because a document editor is only useful if it gets out of the way. If bold, headings, lists, and undo don't behave the way people expect, nobody will trust it with real work, no matter how good the sharing model is.

## Why I thought about collaboration differently

Simultaneous editing can be useful, but I've experienced it becoming messy when multiple people change the same content at once.

**How the idea evolved:**

1. **First idea — controlled editing sessions.** In my planning documents, I wanted people to see when someone else is editing, view the last saved version, and get notified when editing is finished — an alternative to live multi-cursor editing.
2. **Current idea — Collaborative Drafts.** My idea then evolved into independent personal drafts: each person works in their own draft instead of typing over each other in one shared copy. Changes come back through a GitHub-style review, and the team votes on which changes to accept.

I chose a review-and-vote model because it addresses the problem I experienced directly: nobody's work gets overwritten mid-sentence, and the team decides together what goes into the shared document. The detailed rules (who can vote, how ties resolve, how competing drafts merge) are design questions for the next phase; I have not finalized them.

**Why I postponed it:** I did not want to sacrifice reliable document creation, saving, importing, and sharing to build it during a short assignment. My rule for this project is that Collaborative Drafts wait until create, edit, save, import,

share, tests, and a real hosted deployment all work. Those now work, so drafts are the next major feature — not part of this submission.

## Who it is for

Small teams who want to create, format, import, and share documents in a browser without installing software. My plan said two demo users would be enough; the build uses three (Tracy, Nathan, Nora) so owner, editor, and viewer can all be shown at once.

## Primary user journey

1. Open the app and see documents I own versus documents shared with me.
2. Create a document and give it a meaningful title.
3. Enter and format content using a familiar toolbar and working menus.
4. Save, refresh the page, and reopen the same document with formatting preserved.
5. Import a TXT or Markdown file as an editable document.
6. Share a document with another account as a viewer or editor, then copy its link.
7. Switch to the recipient account and verify their allowed actions; verify an unauthorized account cannot access it.

## Scope and status

### P0 — complete these before anything else

| Requirement | Status |
|---|---|
| Create, rename, save, reopen | **Verified** locally and on the live site |
| Delete ("only if there is time") | Not built |
| Bold, italic, underline, headings, bullet and numbered lists | **Verified** |
| Undo and redo | **Verified** |
| Menus with a small, deliberate set of real actions | **Verified** — File, Edit, View, Format. Insert and Tools were left out because they had no working actions yet (see Decisions) |
| Import TXT and MD with visible limits | **Verified** — `.txt`, `.md`, `.markdown`, 500 KB, limits shown in the UI |
| Owner, editor, viewer; owner can grant and revoke | **Verified** locally; grant verified on the live site |
| Copy link that does not bypass access | **Verified** locally |
| Clear owned/shared sections | **Verified** |
| Persistence of content, title, formatting, and grants | **Verified** locally and on Supabase |
| Validation, clear errors, automated tests, hosted deployment | **Verified** — 19 tests; live at https://boogledox-144u.vercel.app |

### P1 — add after the core flow is dependable

| Item | Status |
|---|---|
| Page-like canvas, margins, zoom, visual ruler | Built (zoom is view-only and not saved) |
| Saving/saved/error status | Built and verified |
| Font family/size, color, highlight, alignment, spacing, indentation, checklist | Not started |
| Controlled edit sessions / presence | Replaced by the Collaborative Drafts plan; not started |

### Explicitly out of scope for this submission

- Full concurrent multi-cursor editing.
- Advanced pagination and ruler-based tab stops.
- Comments, suggestions, revision history, and complex exports.
- Full DOCX parsing and enterprise-grade access management.

## Product rules

**Owner:** can view, edit, rename, share, change permissions, and revoke access. **Editor:** can view and edit but cannot change sharing settings. **Viewer:** can read but cannot alter the document. **Unauthorized user:** receives the same "not found" response as a missing document, even with the URL.

A copied link is an address, not a permission grant. A separate "anyone with the link" feature would require an intentional access policy and a revocable token; it should not be silently enabled.

## Decisions and tradeoffs

I chose depth over breadth. The familiar menus matter to the experience, but showing a menu full of nonfunctional buttons would be misleading. I would rather ship fewer menu actions that actually work — which is why Insert and Tools are not in this build even though they were in my original menu list.

I also chose access correctness and persistence ahead of live collaboration. A sharing demo isn't convincing if a viewer can modify the document or if someone can open a restricted document by guessing the URL. That is why permissions are enforced on the server and in the database, not only by hiding buttons.

## What success looks like

A reviewer can open the deployed app, create and format a document, import a file, save it, refresh, share it, and verify the difference between viewer and editor permissions in a few minutes.